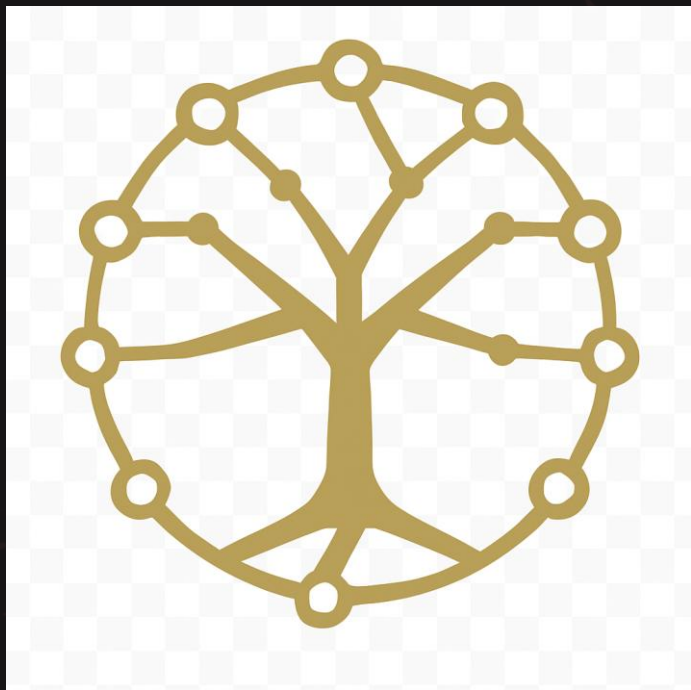




# Автоматическое построение онтологий с помощью LLM как внешний слой памяти, верификации и планирования для гибридных интеллектуальных систем



ОНТОХАБ

Pavel Salovskii · Iuliia Gorshkova

В этой работе рассматривается архитектура, в которой большая языковая модель больше не трактуется как единственный носитель знания. Вместо этого LLM выступает как языковой интерфейс, слой интерпретации и генератор предложений, а долговременная память, проверка фактов и планирование делегируются внешнему онтологическому слою.



## Основной тезис

Знания переносятся из латентного пространства модели в явный, обновляемый и проверяемый граф.



LLM

Ontology

Verification

Planning

На основе статьи:

<https://arxiv.org/abs/2604.20795>

# LLM сильны в языке, но слабы в состоянии

Большие языковые модели демонстрируют впечатляющее качество генерации текста, рассуждений на естественном языке и обобщения. Однако при переходе к задачам, где требуются устойчивое состояние, формальные ограничения и проверяемая память, становятся видны фундаментальные ограничения.

## Ключевые ограничения

- Знания распределены в параметрах модели
- Контекст ограничен окном prompt
- Ответы могут быть правдоподобными, но ложными
- Трудно отделить факт, предположение и галлюцинацию
- Нет явного представления состояния системы

## Главный тезис

LLM не проваливаются прежде всего в языке. Они проваливаются в **состоянии, ограничениях и времени.**



# Почему одного текста недостаточно

Текст является естественной средой человеческой коммуникации, но слабым представлением для машинного reasoning. В тексте содержатся сущности, отношения, правила, исключения, временные зависимости и причинные связи, но эти структуры остаются **неявными**.

Обычная LLM или RAG-система работает с текстовыми фрагментами. Даже если найденный фрагмент релевантен, система всё равно должна восстановить формальную структуру, скрытую внутри текста.



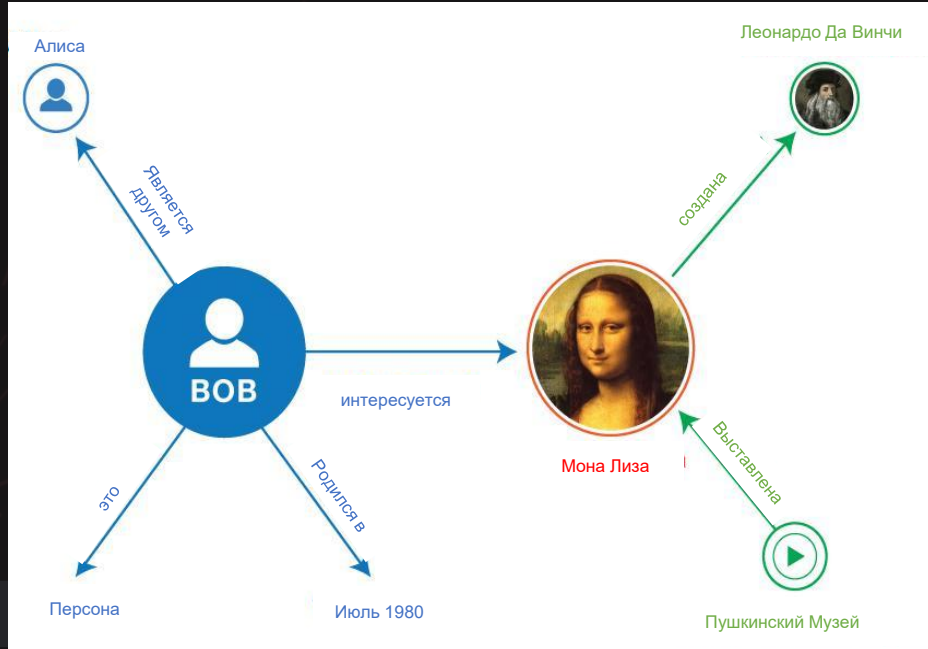
Онтология делает явным то, что существует в предметной области: какие классы объектов допустимы, какие отношения могут их связывать и какие ограничения должны выполняться. Такой переход превращает документы в структурированный и проверяемый слой знания.

# RAG на неструктурированных данных

## – Онтология

```
<body prefix="foaf: http://xmlns.com/foaf/0.1/"
  schema: http://schema.org/"
  dct: http://purl.org/dc/terms/">
  <div resource="http://example.org/foaf" type="foaf:Person">
    <p>
      Bob knows <a property="foaf:knows" href="http://example.org/alice#me">Alice</a>
      and was born on the <time property="schema:birthDate">1990-07-04</time>.
    </p>
    <p>
      Bob is interested in <span property="foaf:topic_interest"
      resource="http://www.wikidata.org/entity/Q12418">the Mona Lisa</span>.
    </p>
  </div>
  <div resource="http://www.wikidata.org/entity/Q12418">
    <p>
      The <span property="dct:term:title" href="http://www.wikidata.org/entity/Q12418">Mona Lisa</span> was painted by
      <a property="dct:term:creator" href="http://dtpedia.org/resource/leonardo_da_Vinci">Leonardo da Vinci</a>
      and is the subject of the video
      <a href="http://data.europeana.eu/item/048802/243FAB618938F417025F17A88B13C5F9A4AD619">"La Joconde à Washington"</a>.
    </p>
  </div>
  <div resource="http://data.europeana.eu/item/048802/243FAB618938F417025F17A88B13C5F9A4AD619">
    <link property="dct:term:subject" href="http://www.wikidata.org/entity/Q12418"/>
  </div>
</body>
```

Онтология сжимает исходные текстовые данные минимум в 10 раз



Источник данных  
Неструктурированный текст из различных источников

1

Построение графа  
Создание семантической сети знаний

2

3

Извлечение сущностей

Автоматическая идентификация ключевых понятий и их взаимосвязей

4

RAG-запросы

Быстрый и точный поиск релевантной информации

Приведенный выше пример показывает, как информация о Леонардо да Винчи и его работе «Мона Лиза» структурируется в онтологический граф. Связи между художником, картиной и музеем явно хранятся, что позволяет немедленно получать ответы на сложные вопросы о культурном наследии.

# LLM как интерфейс, онтология как память

Центральная гипотеза состоит в том, что LLM не должна рассматриваться как самодостаточная система знания. Более надёжная архитектура возникает тогда, когда LLM работает поверх внешней структурированной памяти.

## LLM выполняет

- Интерпретацию запроса
- Генерацию гипотез
- Объяснение результатов
- Оркестрацию инструментов

## Онтология выполняет

- Долговременное хранение знаний
- Явное представление состояния
- Проверку ограничений
- Поддержку планирования и reasoning

**Ключевое утверждение:** LLM генерируют язык. Онтологии поддерживают знание.

# Гибридная LLM-Ontology архитектура

Предлагаемая архитектура объединяет несколько форм памяти и обработки. Пользовательский запрос не передаётся напрямую в LLM как изолированный prompt – система извлекает релевантные данные, строит структурированный контекст, проверяет ограничения и затем генерирует ответ.

## LLM Layer

Генеративный и интерпретационный слой

## Vector RAG

Retrieval на основе семантического сходства

## Ontology Graph

Явное представление знания

## SPARQL

Точный доступ к графу

## SHACL / OWL

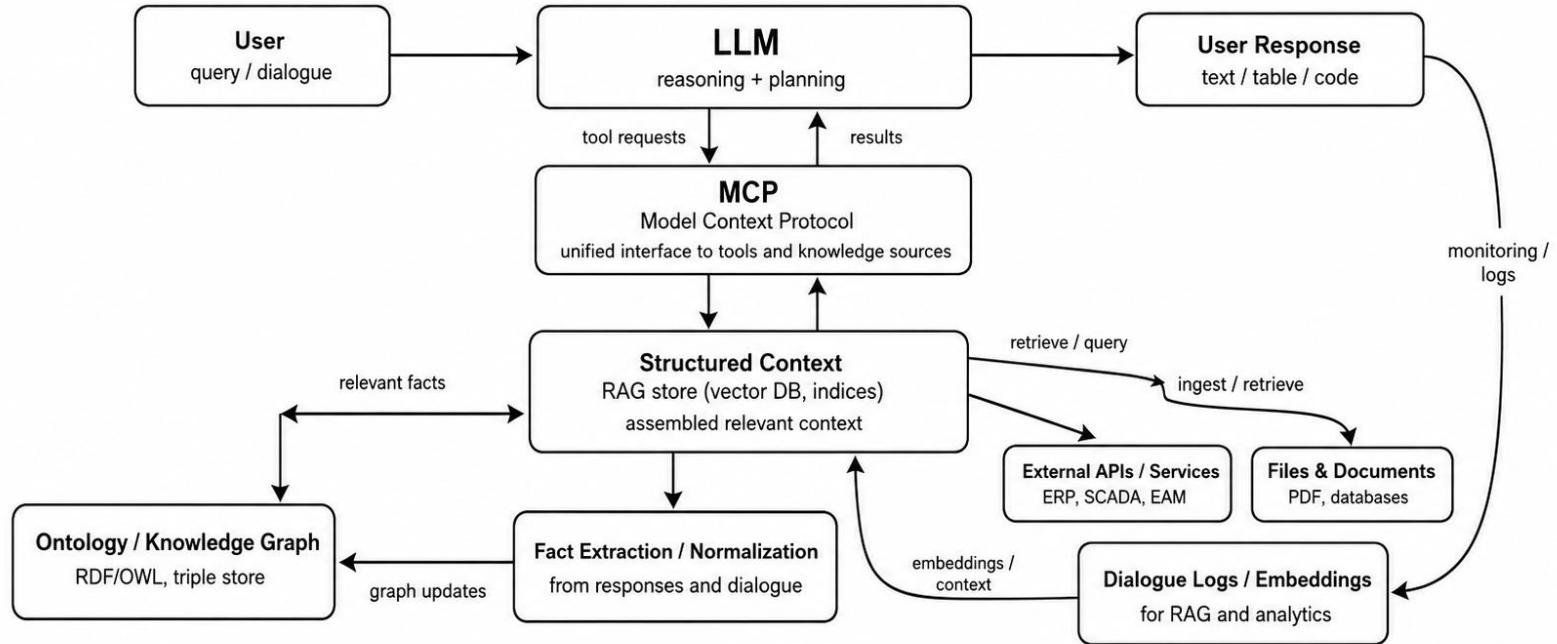
Ограничения и reasoning

## MCP / Tools

Взаимодействие с внешними системами

- ③ Hybrid LLM-Ontology Architecture with MCP Integration. Реконструированная архитектура гибридной системы, объединяющая проектно-специфическую топологию со стандартизированными MCP-компонентами и контекстно-центричной оркестрацией.

# Hybrid LLM–Ontology Architecture with MCP Integration



## MCP ↔ Ontology

- MCP provides unified access to the knowledge graph (read/write via adapters).
- The ontology defines domain semantics and verifiable constraints.
- The LLM uses context from the graph and RAG for accuracy and traceability.

# Векторная память + онтологическая память

Векторная память и онтологическая память решают разные задачи. Векторное хранилище эффективно находит похожие текстовые фрагменты. Онтологическая память хранит явные сущности и отношения — она отвечает не только на вопрос «что похоже?», но и на вопросы «что связано?», «чем ограничено?» и «что из этого следует?»

| Компонент      | Роль                              | Ограничение               |
|----------------|-----------------------------------|---------------------------|
| Vector Store   | Retrieval по сходству             | Нет формальной логики     |
| Ontology Graph | Семантика, отношения, ограничения | Требует нормализации      |
| LLM            | Интерпретация и генерация         | Не гарантирует истинность |
| SHACL / OWL    | Валидация и вывод                 | Может добавлять задержку  |

**Ключевое утверждение:** Векторная память извлекает. Онтологическая память понимает структуру.



# RAG улучшает доступ к тексту, но не создаёт знание

Классический RAG решает важную инженерную проблему: он позволяет LLM использовать внешние документы без дообучения. Однако его слабость в том, что он в основном извлекает текстовые фрагменты. Даже если найденный фрагмент релевантен, система всё ещё должна вывести структуру самостоятельно.

## Что RAG не делает

- Поиск по сходству не является reasoning
- Близость embedding не гарантирует истинность
- Извлечённый контекст может быть неполным
- Зависимости между документами теряются
- Нет формального слоя валидации

### RAG спрашивает

«Какой текст похож?»

### Ontology-RAG спрашивает

«Какие факты и отношения применимы?»

# От ручной инженерии знаний к LLM-assisted ontology construction

Классическая онтологическая инженерия исторически требовала ручной работы экспертов: выделения классов, отношений, правил, ограничений и схем. Это делало онтологии точными, но дорогими и медленными в построении. LLM меняют эту ситуацию — модель может использоваться как первичный извлекатель структуры из неструктурированных и полуструктурированных данных.

## Задача Ontology Builder

Автоматически преобразовывать потоки данных в граф знаний: стоимость построения онтологий, а онтологии повышают надёжность LLM.

01

---

### Обнаружение сущностей

Извлечение именованных объектов из текста

02

---

### Типизация и отношения

Назначение типов, извлечение связей

03

---

### RDF-триплеты и нормализация

Построение, нормализация идентификаторов

04

---

### Валидация и обновление графа

SHACL-проверка и запись в граф

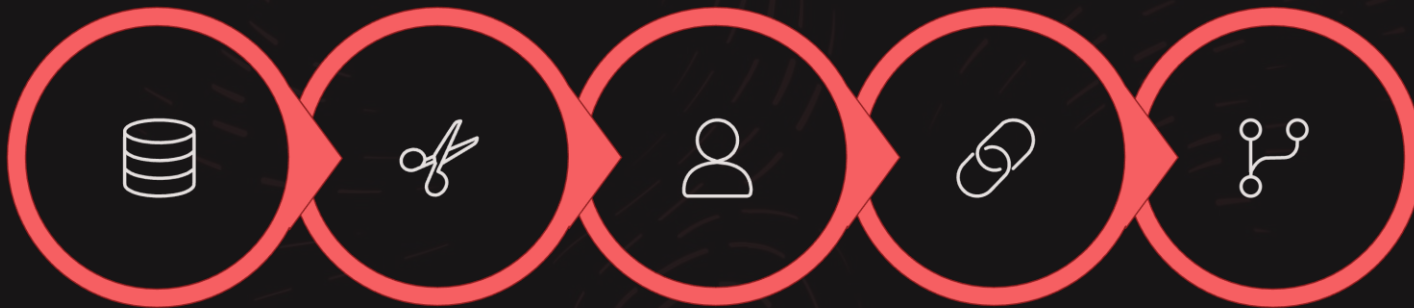
## Функциональное сравнение компонентов

| Компонент               | Основная функция                         | Тип знания                         | Что проверяет / усиливает                       |
|-------------------------|------------------------------------------|------------------------------------|-------------------------------------------------|
| Векторное RAG-хранилище | Быстрый recall по близости               | Распределённые embedding'и         | Релевантные фрагменты текста                    |
| RDF/OWL-граф            | Структурированная память                 | Сущности, отношения, типы, аксиомы | Типизацию, связи, логические следствия          |
| SPARQL                  | Доступ к графу                           | Graph patterns, paths, filters     | Точные graph-aware запросы                      |
| SHACL                   | Валидация графа                          | Shape constraints                  | Структурную корректность и полноту              |
| Reasoning engine        | Классификация и логический вывод         | Аксиомы и правила                  | Непротиворечивость, импликации                  |
| MCP                     | Оркестрация инструментов и контекста     | Protocols + tool schemas           | Композицию workflow и доступ к внешним системам |
| Логи / эмбединги        | Аудиторский след и дополнительная память | Traces, histories, dense vectors   | Воспроизводимость и retracing                   |
| Слой агентов            | Исполнение и специализация               | Планы, tool policies, validators   | Action loop и многошаговое управление           |



# Построение онтологии как feedback pipeline

Данные поступают из гетерогенных источников: текстовых файлов, PDF, баз данных, API и логов диалогов. Они сегментируются, проходят извлечение сущностей, типизацию, извлечение отношений и нормализацию. Затем генерируются RDF-триплеты, которые валидируются с использованием SHACL/OWL-ограничений и записываются в графовую базу данных.



Данные

Сегментация

Сущности

Отношения

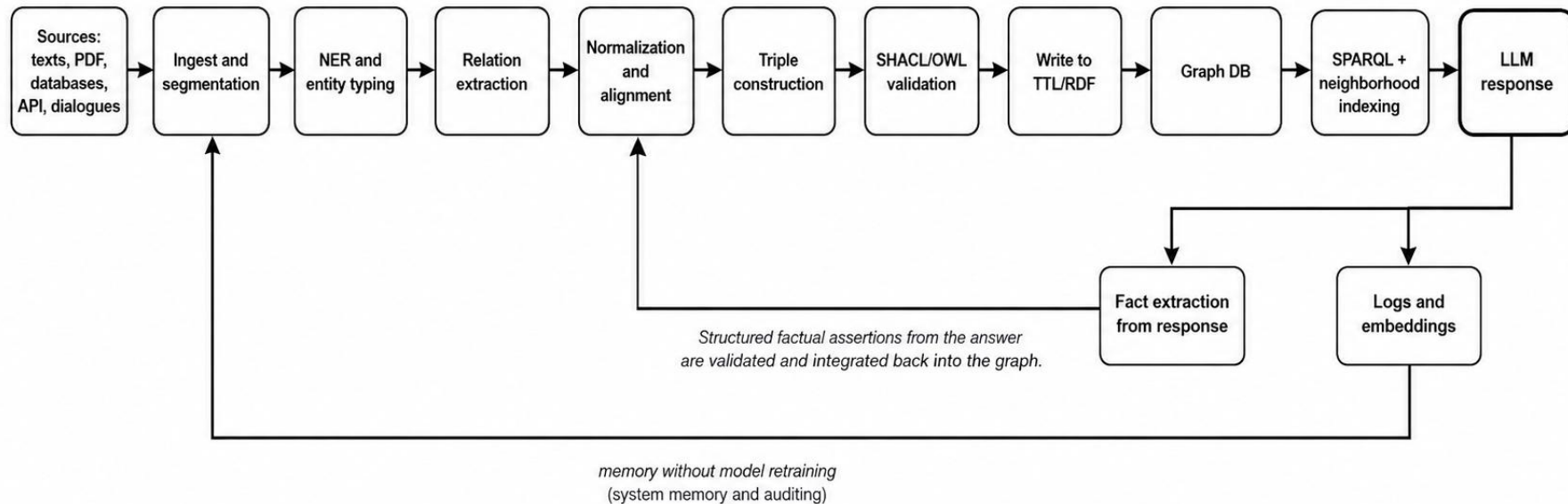
RDF триплеты



**Ontology Builder Pipeline.** Цикл обратной связи критически важен: ответы LLM также анализируются – из сгенерированных ответов извлекаются фактические утверждения, которые после валидации интегрируются в граф. Система развивает память без переобучения модели.

# Ontology Builder Pipeline

*Closed-loop knowledge graph construction with feedback from LLM output*



# Формальный стек онтологической памяти

Онтологический слой строится на стандартах Semantic Web. Каждый уровень стека выполняет строго определённую функцию.

## RDF

Представляет знание в виде триплетов:

`subject — predicate — object`

Пример: `Robot — performs — Task`

## OWL

Расширяет RDF классами, иерархиями, свойствами и логическим выводом. Позволяет представлять не только факты, но и правила предметной области.

## SHACL

Валидирует ограничения: какие данные допустимы, какие поля обязательны, какие типы значений корректны и какие отношения запрещены.

**RDF** хранит факты. **OWL** определяет смысл. **SHACL** проверяет корректность.



# Устойчивое структурированное состояние

В отличие от контекстного окна LLM, онтология сохраняет знания между сессиями. Она может накапливать факты, обновлять отношения, хранить изменения и поддерживать единое состояние предметной области.

## Что должен помнить агент / робот

- Где расположены объекты
- Какие действия уже выполнены
- Какие ограничения активны
- Какие цели остаются нерешёнными
- Какие ошибки происходили ранее



### Ключевое утверждение

Память — это не хранение текста.  
Память — это устойчивое структурированное состояние.  
Агент не просто вспоминает фрагменты прошлого — он использует накопленную структуру для принятия решений.

# От правдоподобия к проверяемости

Одна из главных проблем LLM — правдоподобные, но ложные ответы. Модель может уверенно сгенерировать утверждение, которое не поддерживается источниками или противоречит известным данным. **Fact Analyzer** решает это через проверку сгенерированных утверждений относительно онтологического графа.



## Supported

Утверждение подтверждается онтологическим графом



## Contradicted

Утверждение конфликтует с данными графа



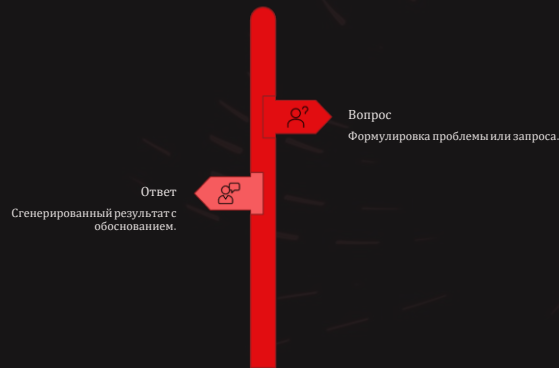
## Unknown

Недостаточно данных для верификации

**Ключевое утверждение:** Система должна не только отвечать. Она должна знать, подтверждён ли ответ.

# Generation → Verification → Correction

## Классический процесс



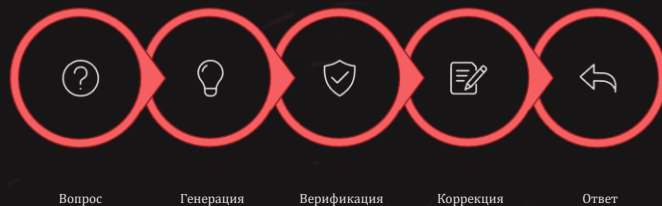
## Значение для safety-critical systems

В критических системах недостаточно просто выдать ответ. Система должна также знать:

- На чём основан ответ
- Какие ограничения были применены
- Какие факты подтверждены
- Где система находится в состоянии неопределённости

⚠ Во время генерации LLM создаёт предварительный ответ. Затем система извлекает фактические утверждения и проверяет их через онтологию, SHACL-ограничения и доступные источники. При обнаружении противоречия ответ корректируется или помечается как неподтверждённый.

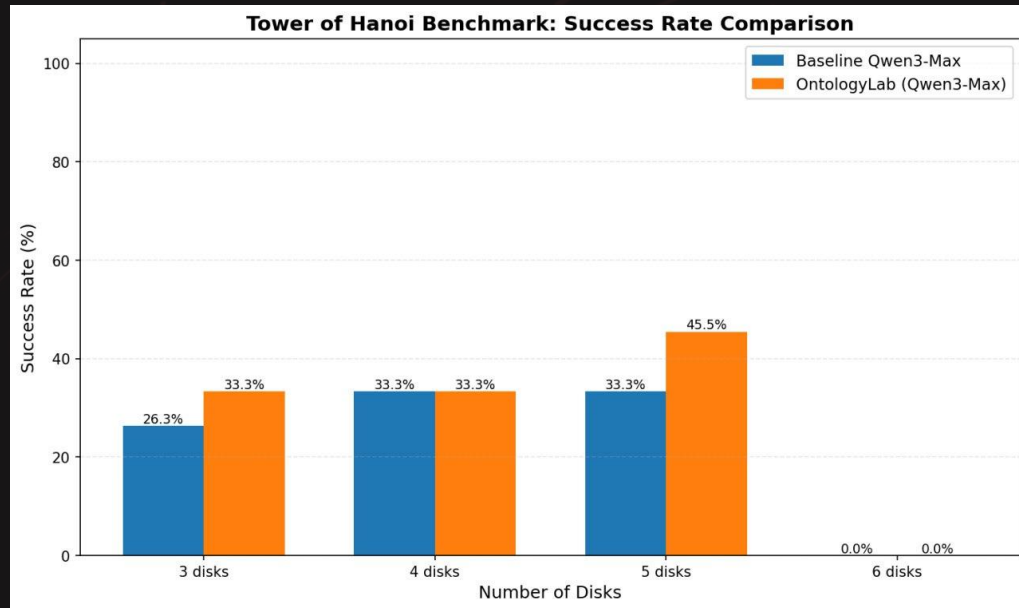
## Расширенный процесс верификации



# Прохождения теста на понимание устройства мира с помощью онтологии в сравнении с ЛЛМ моделью QWEN-3 MAX

Тест «Ханойская башня» используется как классический бенчмарк для оценки способности системы к многошаговому логическому планированию. На графике показано сравнение базовой модели и версии, дополненной онтологическим слоем. Базовая модель решает задачу частично, опираясь на вероятностное рассуждение, поэтому её успех ограничен: около 26% для трёх дисков и около 33% для четырёх и пяти дисков. Онтологический слой улучшает результат, поскольку добавляет формальное описание правил задачи (диск нельзя класть на меньший, перемещается только один диск за ход, состояние башен описывается как структура). Благодаря этому система может строить более последовательный план действий, что повышает успешность решения — до ~33% для трёх дисков и ~45% для пяти дисков.

С точки зрения архитектуры ИИ это демонстрирует ключевое преимущество онтологического подхода: модель перестаёт угадывать последовательность действий и начинает опираться на формализованную модель мира и правил задачи. Онтология задаёт структуру объектов (диски, стержни, состояние башен) и ограничения операций, что позволяет системе проверять допустимость каждого шага и строить корректную цепочку действий. В результате сложные задачи планирования решаются стабильнее, а вероятность ошибок, связанных с «галлюцинациями» модели, существенно снижается.



# Эффект проявляется в зоне средней сложности

Результаты показывают, что онтологический слой улучшает качество планирования в задачах средней сложности. Улучшения наблюдаются для 3 и 5 дисков, тогда как для 6 дисков обе системы не справляются.

## Интерпретация результатов

- **Онтология сохраняет состояние**  
Явный граф удерживает позиции дисков между шагами
- **SHACL/OWL отклоняет недопустимые шаги**  
Ограничения задачи становятся формальными правилами
- **Граф делает правила явными**  
Символическое сопровождение помогает при средней сложности
- **Слабые места остаются**  
Полнота ограничений и точность перевода LLM-to-graph

## Ключевое утверждение

Онтология помогает сильнее всего тогда, когда задача уже слишком структурна для чистого языка, но ещё не слишком сложна для символического сопровождения.

# Почему это важно для роботов

В робототехнике LLM редко терпят неудачу потому, что не могут породить язык. Основные ошибки возникают из-за неверного состояния мира, частичной наблюдаемости, шума сенсоров, временных задержек и нарушения физических ограничений.

## Что должен знать робот

- Где расположены объекты
- Какие действия разрешены
- Какие действия небезопасны
- Как меняется состояние после действий
- Какие цели и подцели активны

## Роль онтологии

Онтология выступает как внешняя модель мира: определяет типы объектов, допустимые действия, ограничения среды и отношения между состояниями.



### Важное ограничение

Классические RDF/OWL/SHACL предполагают относительно чистые данные. Реальные роботы работают в условиях шума и неопределённости. Следующий этап — **probabilistic ontology** и **incomplete-state handling**.



# Эволюция архитектур памяти AI

Современные AI-системы движутся от простого retrieval к более структурированным формам памяти. Долгосрочное направление — не увеличение prompt, а улучшение внешнего состояния.



## RAG

Извлечение релевантных текстовых фрагментов из внешних документов



## GraphRAG

Использование структуры графа для улучшения retrieval



## Ontology-RAG

Добавление типов, правил, ограничений и логического вывода



## World Models

Представление состояния мира и переходов между состояниями



## Hybrid Cognitive Systems

Интеграция LLM, онтологий, планирования, действия и обратной связи

# От post-hoc verification к constraint-aware generation

Текущая архитектура уже может проверять ответы после генерации. Однако для safety-critical systems этого недостаточно. Ограничения должны участвовать в самом процессе генерации.

## Направления исследований

Probabilistic ontologies

Uncertainty-aware reasoning

Incomplete-state representation

Ontology-aware decoding

Integration with robotics planners

Continuous graph updates

Benchmark suite for ontology-augmented agents

## Главный исследовательский переход

Текущий подход

Generate → then verify

Целевой подход

Generate under constraints

Это позволит не только исправлять ошибки после их появления, но и снижать вероятность генерации недопустимых планов изначально.

# Что демонстрирует работа

1

LLM не является полноценной системой памяти

Параметры модели и контекстные окна не обеспечивают надёжное долговременное хранение знания.

2

RAG полезен, но недостаточен

RAG улучшает доступ к тексту, но не создаёт формальную модель предметной области.

3

Онтология добавляет структуру

RDF/OWL/SHACL могут хранить факты, определять смысл и валидировать ограничения.

4

LLM могут строить онтологии автоматически

Это снижает стоимость knowledge engineering и делает возможными самообновляемые системы знания.

5

Гибридная архитектура повышает надёжность

Разделение генерации, памяти, верификации и планирования делает систему более объяснимой и пригодной для критических применений.

LLM производит гипотезы. Онтология ограничивает, проверяет и запоминает.

## Ключевые источники и ссылки

1. W3C. RDF 1.1 Concepts and Abstract Syntax. W3C Recommendation, 2014. [35]
2. W3C. OWL 2 Web Ontology Language Document Overview. W3C Recommendation, 2012. [36]
3. W3C. SPARQL 1.1 Query Language. W3C Recommendation, 2013. [37]
4. W3C. Shapes Constraint Language (SHACL). W3C Recommendation, 2017. [38]
5. Model Context Protocol Specification. MCP specification, version dated 2025-11 25. [16]
6. D. Edge et al. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv:2404.16130, 2024. [39]
7. H. Han et al. Retrieval-Augmented Generation with Graphs (GraphRAG). arXiv:2501.00309, 2025. [40]
8. Welcome — GraphRAG. Microsoft Research documentation, 2025. [41]
9. H. Bian. LLM-empowered knowledge graph construction: A survey. arXiv:2510.20345, 2025. [42]
10. D. Garijo et al. LLMs for Ontology Engineering: A landscape of Tasks and Benchmarking challenges. CEUR Workshop Proceedings, 2025. [43]
11. V. K. Kommineni, B. König-Ries, S. Samuel. From human experts to machines: An LLM supported approach to ontology and knowledge graph construction. arXiv:2403.08345, 2024. [44]
12. A. S. Lippolis et al. Ontology Generation using Large Language Models. arXiv:2503.05388, 2025. [45]
13. M. Nayyeri et al. Retrieval-Augmented Generation of Ontologies from Relational Databases. arXiv:2506.01232, 2025. [46]
14. X. Feng, X. Wu, H. Meng. Ontology-grounded Automatic Knowledge Graph Construction by LLM under Wikidata schema. arXiv:2412.20942, 2024. [47]
15. T. Aggarwal, A. Salatino, F. Osborne, E. Motta. Large Language Models for Scholarly Ontology Generation: An Extensive Analysis in the Engineering Field. arXiv:2412.08258, 2024/2025. [48]
16. H. V. Khurdula, V. Agarwal, Y. D. Khemlani. Interfaze: The Future of AI is built on Task-Specific Small Models. arXiv:2602.04101, 2026. [49]
17. P. Belcak et al. Small Language Models are the Future of Agentic AI. arXiv:2506.02153, 2025. [50]
18. S. Kambhampati et al. LLMs Can't Plan, But Can Help Planning in LLM-Modulo Frameworks. arXiv:2402.01817, 2024. [51]
19. K. Valmeekam et al. Large Language Models Still Can't Plan (A Benchmark for LLMs on Planning and Reasoning about Change). FMDM@NeurIPS / OpenReview, 2022.

# Ваши вопросы?

Павел Саловский  
Директор по  
Искусственному  
интеллекту



[pavel.salovskii@ontohub.ru](mailto:pavel.salovskii@ontohub.ru)

[+7 926 106 97 68](tel:+79261069768)

[Telegram: @pasalovs](https://t.me/pasalovs)

<https://ontohub.ru/>